



Modeling with UML

"Every mechanic is familiar with the problem of the part you can't buy because you can't find it because the manufacturer considers it a part of something else."

—Robert Pirsig, in *Zen and the Art of Motorcycle Maintenance*.

Notations enable us to articulate complex ideas succinctly and precisely. In projects involving many participants, often of different technical and cultural backgrounds, accuracy and clarity are critical as the cost of miscommunication increases rapidly.

For a notation to enable accurate communication, it must come with a *well-defined* semantics, it must be *well suited* for representing a given aspect of a system, and it must be *well understood* among project participants. In the latter lies the strength of standards and conventions: When a notation is used by a large number of participants, there is little room for misinterpretation and ambiguity. Conversely, when many dialects of a notation exists, or when a very specialized notation is used, the notation users are prone to misunderstandings as each user imposes its own interpretation. We selected UML (Unified Modeling Language, [OMG, 1998]) as a primary notation for this book, given that it has a well-defined semantics, provides a spectrum of notations for representing different aspects of a system, and has been accepted as a standard notation in the industry.

In this chapter, we first describe the concepts of modeling in general and object-oriented modeling in particular. We then describe five fundamental notations of UML that we use throughout the book: use case diagrams, class diagrams, sequence diagrams, statechart diagrams, and activity diagrams. For each of these notations, we describe its basic semantics and provide examples. We revisit these notations in detail in later chapters as we describe the activities that use them. Specialized notations that we use in only one chapter are introduced later, such as PERT charts in Chapter 11, *Project Management*, and UML component and deployment diagrams in Chapter 6, *System Design*.

2.1 Introduction

UML is a notation that resulted from the unification of OMT (Object Modeling Technique, [Rumbaugh et al., 1991]), Booch [Booch, 1994], and OOSE (Object-Oriented Software Engineering, [Jacobson et al., 1992]). UML has also been influenced by other object-oriented notations, such as those introduced by Shlaer/Mellor [Mellor & Shlaer, 1998], Coad/Yourdon [Coad et al., 1995], Wirfs-Brock [Wirfs-Brock et al., 1990], and Martin/Odell [Martin & Odell, 1992]. UML has been designed for a broad range of applications. Hence, it provides constructs for a broad range of systems and activities (e.g., real-time systems, distributed systems, analysis, system design, deployment). System development focuses on three different models of the system:

- The **functional model**, represented in UML with use case diagrams, describes the functionality of the system from the user's point of view.
- The **object model**, represented in UML with class diagrams, describes the structure of a system in terms of objects, attributes, associations, and operations.
- The **dynamic model**, represented in UML with sequence diagrams, statechart diagrams, and activity diagrams, describes the internal behavior of the system. Sequence diagrams describe behavior as a sequence of messages exchanged among a *set of objects*, whereas statechart diagrams describe behavior in terms of states of an *individual object* and the possible transitions between states.

In this chapter, we describe UML diagrams for representing these models. Introducing these notations represents an interesting challenge. On the one hand, understanding the purpose of a notation requires some familiarity with the activities that use it. On the other hand, it is necessary to understand the notation before describing the activities. To address this issue, we introduce UML iteratively. In the next section, we first provide an overview of the five basic notations of UML. In Section 2.3, we introduce the fundamental ideas of modeling. In Section 2.4, we revisit the five basic notations of UML in light of modeling concepts. In subsequent chapters, we examine these notations in detail when we introduce the activities that use them.

2.2 An overview of UML

In this section, we briefly introduce five UML notations:

- use case diagrams (Section 2.2.1)
- class diagrams (Section 2.2.2)
- sequence diagrams (Section 2.2.3)
- statechart diagrams (Section 2.2.4)
- activity diagrams (Section 2.2.5)

2.2.1 Use case diagrams

Use cases are used during requirements elicitation and analysis to represent the functionality of the system. Use cases focus on the behavior of the system from an external point of view. A use case describes a function provided by the system that yields a visible result for an actor. An actor describes any entity that interacts with the system (e.g., a user, another system, the system's physical environment). The identification of actors and use cases results in the definition of the boundary of the system, that is, in differentiating the tasks accomplished by the system and the tasks accomplished by its environment. The actors are outside the boundary of the system, whereas the use cases are inside the boundary of the system.

For example, Figure 2-1 depicts a use case diagram for a simple watch. The *WatchUser* actor may either consult the time on their watch (with the *ReadTime* use case) or set the time (with the *SetTime* use case). However, only the *WatchRepairPerson* actor may change the battery of the watch (with the *ChangeBattery* use case).

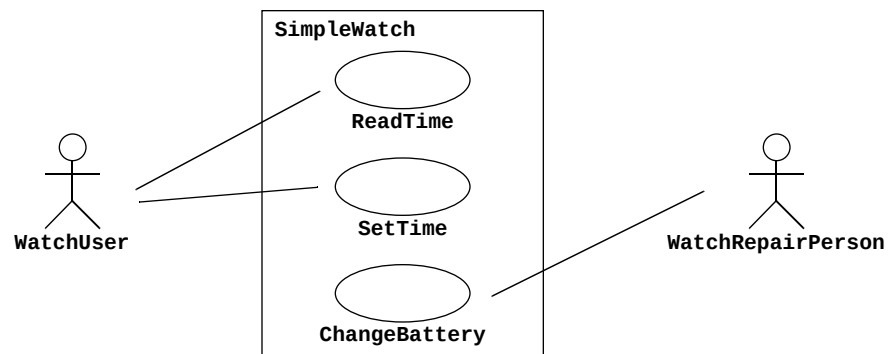


Figure 2-1 A UML use case diagram describing the functionality of a simple watch. The *WatchUser* actor may either consult the time on her watch (with the *ReadTime* use case) or set the time (with the *SetTime* use case). However, only the *WatchRepairPerson* actor may change the battery of the watch (with the *ChangeBattery* use case). Actors are represented with stick figures, use cases with ovals, and the boundary of the system with a box enclosing the use cases.

2.2.2 Class diagrams

We use class diagrams to describe the structure of the system. Classes are abstractions that specify the common structure and behavior of a set of objects. Objects are instances of classes that are created, modified, and destroyed during the execution of the system. Objects have state that includes the values of its attributes and its relationships with other objects.

Class diagrams describe the system in terms of objects, classes, attributes, operations, and their associations. For example, Figure 2-2 is a class diagram describing the elements of all the watches of the *SimpleWatch* class. These watch objects all have an association to an object of the *PushButton* class, an object of the *Display* class, an object of the *Time* class, and an object

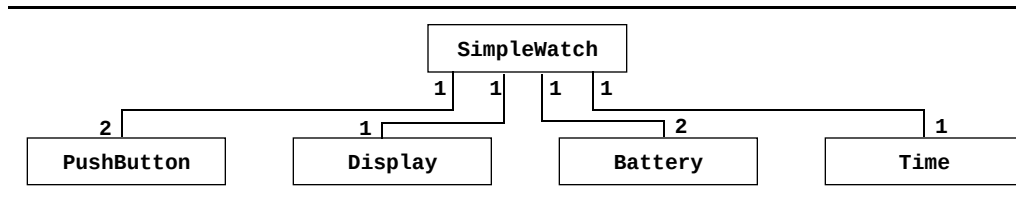


Figure 2-2 A UML class diagram describing the elements of a simple watch.

of the **Battery** class. The numbers on the ends of associations denote the number of links each **SimpleWatch** object can have with an object of a given class. For example, a **SimpleWatch** has exactly two **PushButton**s, one **Display**, two **Batteries**, and one **Time**. Similarly, all **PushButton**, **Display**, **Time**, and **Battery** objects are associated to exactly one **SimpleWatch** object.

2.2.3 Sequence diagrams

Sequence diagrams are used to formalize the behavior of the system and to visualize the communication among objects. They are useful for identifying additional objects that participate in the use cases. We call objects involved in a use case **participating objects**. A sequence diagram represents the interactions that take place among these objects. For example, Figure 2-3 is a sequence diagram for the **SetTime** use case of our simple watch. The leftmost column represents the **WatchUser** actor who initiates the use case. Labeled arrows represent stimuli that an actor or an object sends to other objects. In this case, the **WatchUser** presses button 1 twice and button 2 once to set her watch a minute ahead. The **SetTime** use case terminates when the **WatchUser** presses both buttons simultaneously.

2.2.4 Statechart diagrams

Statechart diagrams describe the behavior of an individual object as a number of states and transitions between these states. A state represents a particular set of values for an object. Given a state, a transition represents a future state the object can move to and the conditions associated with the change of state. For example, Figure 2-4 is a statechart diagram for the **SimpleWatch**. Note that this diagram represents different information than the sequence diagram of Figure 2-3 does. The sequence diagram focuses on the messages exchanged between objects as a result of external events created by actors. The statechart diagram focuses on the transitions between states as a result of external events for an individual object.

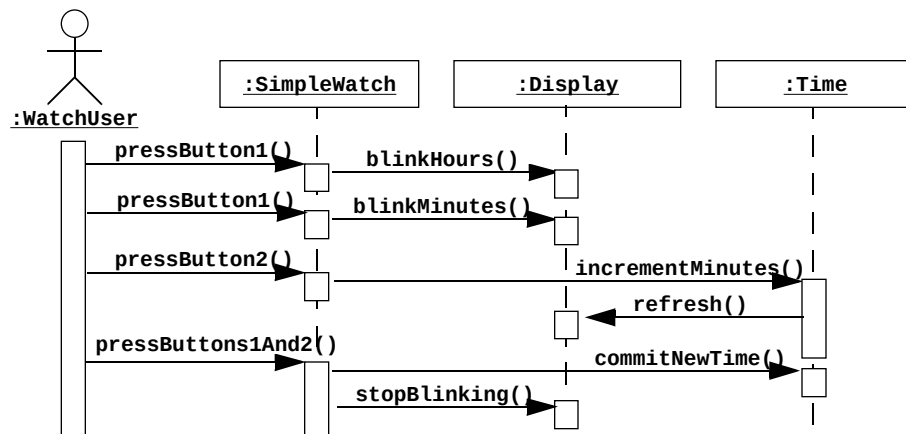


Figure 2-3 A UML sequence diagram for the SimpleWatch. The leftmost column represents the timeline of the WatchUser actor who initiates the use case. The other columns represent the timeline of the objects that participate in this use case. Object names are underlined to denote that they are instances (as opposed to classes). Labeled arrows are stimuli that an actor or an object sends to other objects.

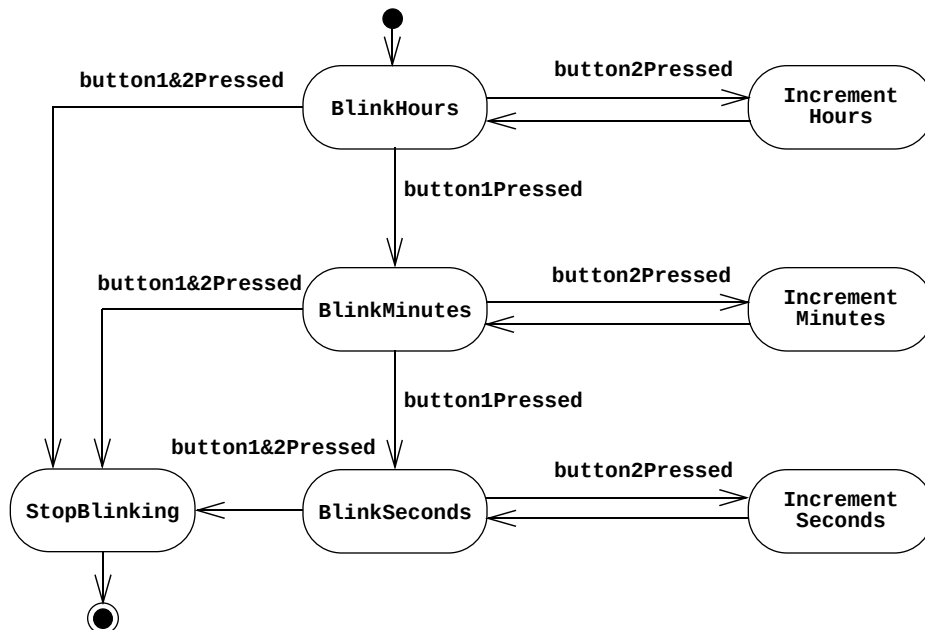


Figure 2-4 A UML statechart diagram for SetTime use case of the SimpleWatch.

2.2.5 Activity diagrams

An activity diagram describes a system in terms of activities. Activities are states that represent the execution of a set of operations. The completion of these operations triggers a transition to another activity. Activity diagrams are similar to flowchart diagrams in that they can be used to represent control flow (i.e., the order in which operations occur) and data flow (i.e., the objects that are exchanged among operations). For example, Figure 2-5 is an activity diagram representing activities related to managing an Incident in FRIEND. Rounded rectangles represent activities; arrows represent transitions between activities; thick bars represent the synchronization of the control flow. The activity diagram of Figure 2-5 depicts that the `AllocateResources`, `CoordinateResources`, and `DocumentIncident` can be initiated only after the `OpenIncident` activity has been completed. Similarly, the `ArchiveIncident` activity can be initiated only after the completion of `AllocateResources`, `CoordinateResources`, and `DocumentIncident`. These latter three activities, however, can occur concurrently.

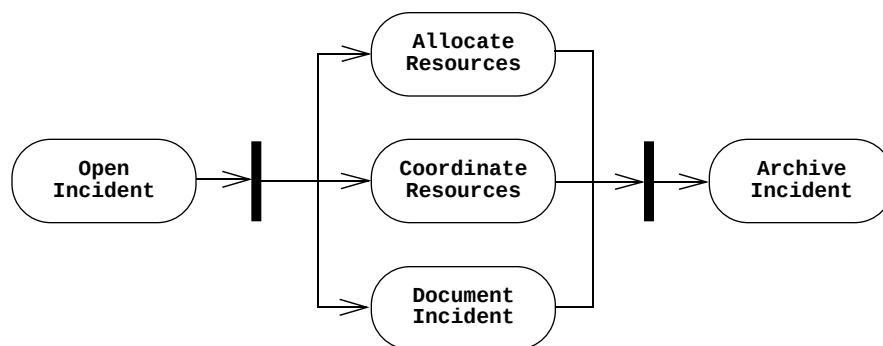


Figure 2-5 An example of a UML activity diagram. Activity diagrams represent behavior in terms of activities and their precedence constraints. The completion of an activity triggers an outgoing transition, which in turn may initiate another activity.

This concludes our first walkthrough of the five basic notations of UML. Now, we go into more detail: In Section 2.3, we introduce basic modeling concepts, including the definition of systems, models, types, and instances, abstraction, and falsification. In Sections 2.4.1–2.4.5, we describe in detail use case diagrams, class diagrams, sequence diagrams, statechart diagrams, and activity diagrams. We illustrate their use with a simple example. Section 2.4.6 describes miscellaneous constructs, such as packages and notes, that are used in all types of diagrams. We use these five notations throughout the book to describe software systems, work products, activities, and organizations. By the consistent and systematic use of a small set of notations, we hope to provide the reader with an operational knowledge of UML.

2.3 Modeling concepts

In this section, we describe the basic concepts of modeling. We first define the terms **system** and **model** and discuss the purpose of **modeling**. We then define the terms **concept** and **phenomenon**. We explain their relationship to programming languages and terms such as **types**, **classes**, **instances**, and **objects**. Finally, we describe how object-oriented modeling focuses on building an abstraction of the system environment as a basis for the system model.

2.3.1 Systems, models, and views

A **system** is an organized set of communicating parts designed for a specific purpose. A car, composed of four wheels, a chassis, a body, and an engine, is designed to transport people. A watch, composed of a battery, a circuit, wheels, and hands, is designed to measure time. A payroll system, composed of a mainframe computer, printers, disks, software, and the payroll staff, is designed to issue salary checks for employees of a company. Parts of a system can in turn be considered as simpler systems called **subsystems**. The engine of a car, composed of cylinders, pistons, an injection module, and many other parts, is a subsystem of the car. Similarly, the integrated circuit of a watch and the mainframe computer of the payroll system are subsystems. This subsystem decomposition can be recursively applied to subsystems. Objects represent the end of this recursion, when each piece is simple enough that we can fully comprehend it without further decomposition.

Many systems are made of numerous subsystems interconnected in complicated ways, often so complex that no single developer can manage its entirety. **Modeling** is a means for dealing with this complexity. Complex systems are generally described by more than one model, each focusing on a different aspect or level of accuracy. Modeling means constructing an abstraction of a system that focuses on interesting aspects and ignores irrelevant details. What is interesting or irrelevant varies with the task at hand. For example, assume we want to build an airplane. Even with the help of field experts, we cannot build an airplane from scratch and hope that it will function correctly on its maiden flight. Instead, we first build a scale model of the air frame to test its aerodynamic properties. In this scale model, we only need to represent the exterior surface of the airplane. We can ignore details such as the instrument panel or the engine. In order to train pilots for this new airplane, we also build a flight simulator. The flight simulator needs to accurately represent the layout and behavior of flight instruments. In this case, however, details about the exterior of the plane can be ignored. Both the flight simulator and the scale model are much less complex than the airplane they represent. Modeling allows us to deal with complexity through a divide-and-conquer approach: For each type of problem we want to solve (e.g., testing aerodynamic properties, training pilots), we build a model that only focuses on the issues relevant to the problem. Generally, modeling focuses on building a model that is simple enough for a person to grasp completely. A rule of thumb is that each entity should contain at most 7 ± 2 parts [Miller, 1956].

Unfortunately, even a model may become so complex that it is not easily understandable. As with systems, we apply the same divide-and-conquer approach to deal with the complexity of

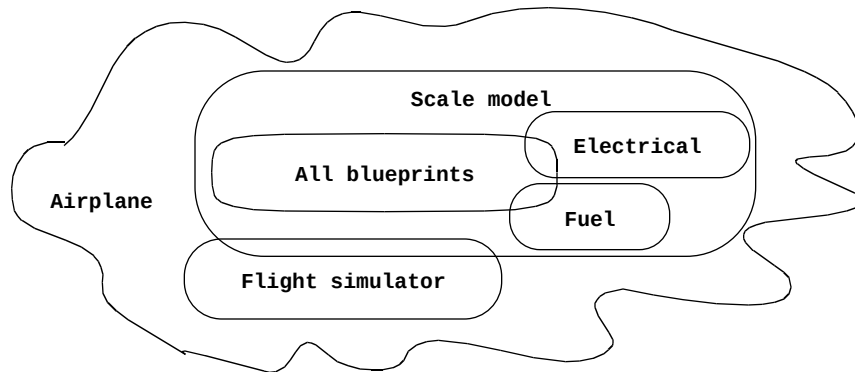


Figure 2-6 A model is an abstraction describing a subset of a system. A view depicts selected aspects of a model. Views and models of a single system may overlap each other.

models. A **view** focuses on a subset of a model to make it understandable (Figure 2-6). For example, all the blueprints necessary to construct an airplane constitute a model. Excerpts necessary to explain the functioning of the fuel system constitute the fuel system view. Views may overlap: A view of the airplane representing the electrical wiring also includes the wiring for the fuel system.

Notations are graphical or textual rules for representing views. A UML class diagram is a graphical view of the object model. In wiring diagrams, each connected line represents a different wire or bundle of wires. In UML class diagrams, a rectangle with a title represents a class. A line between two rectangles represents a relationship between the two corresponding classes. Note that different notations can be used to represent the same view (Figures 2-7 and 2-8).

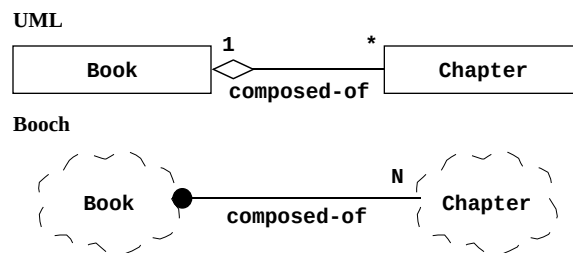


Figure 2-7 Example of describing a model with two different notations. The model includes two classes, Book and Chapter, with the relationship, Book is composed of Chapters. In UML, classes are depicted by rectangles and aggregation associations by a line terminated with a diamond. In the Booch notation, classes are depicted by clouds, and aggregation associations are depicted with a line terminated with a solid circle.

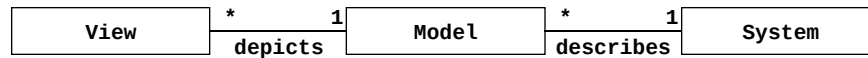


Figure 2-8 Example of describing the same model with two different notations. This UML diagram represents the information of Figure 2-6: A System can be represented by many different Models that can be depicted by many different Views.

In software development, there are also many other notations for modeling systems. UML describes a system in terms of classes, events, states, interactions, and activities. Data flow diagrams [De Marco, 1978] depict how data is retrieved, processed, and stored. Each notation is tailored for a different problem.

In the next sections, we focus in more detail on the process of modeling. We examine the definitions of **concept** and **phenomenon** and their relationship to the programming concepts of **type**, **variable**, **class**, and **object**.

2.3.2 Concepts and phenomena

A **phenomenon** is an object of the world as it is perceived. The following are phenomena:

- this book
- the current savings interest rate is 3%
- my black watch
- Valley Fishermen's Club

A **concept** is an abstraction describing a set of phenomena. The following are concepts:

- textbooks on object-oriented software engineering
- savings interest rates
- black watches
- fishermen's clubs

A concept describes the properties that are common across a set of phenomena. For example, the concept *black watches* is only concerned with the color of watches, not their origin or their quality. A concept is defined as a three-tuple: its **name** (to distinguish it from other concepts), its **purpose** (the properties that determine if a phenomenon is part of the concept or not), and its **members** (the set of phenomena that are part of the concept).¹ Figure 2-9 illustrates the concept of clock. *Clock* is the name of the concept. *Measure time* is the purpose of a clock. *My wrist watch* and *the wall clock above my desk* are members of the clock concept. A club has a name (e.g., "Valley Fishermen's club"), attributes that members must satisfy to be part of the

1. The three components of a concept are also referred as the name, the intension, and the extension.

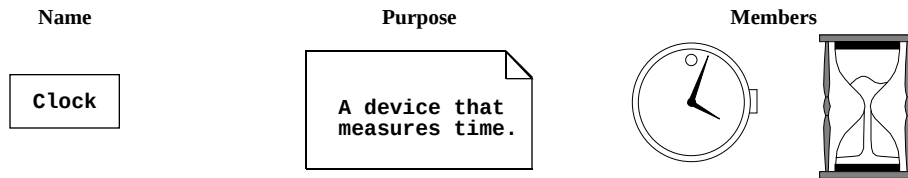


Figure 2-9 The three components of the Clock concept: name, purpose, and members.

club (e.g., “fishermen who live in the Valley”), and actual members (e.g., “John Smith,” “James Doe”).

Abstraction is the classification of phenomena into concepts. **Modeling** is the development of abstractions that can be used to answer specific questions about a set of phenomena. An abstraction is simpler to manipulate and to examine than its corresponding set of phenomena because it contains less information: Irrelevant details are abstracted away. In chemistry, the table of elements summarizes the different types of atoms based on their atomic weight and number of electron pairs. Details such as the availability of each substance, its participation in different molecules, are not represented. In biology, species are classified into family trees based on significant features (e.g., species that are warm blooded, species that have vertebrae). A tree of species ignores issues related to behavior or habitat. In astronomy, stars are classified into different types based on their spectrum and dissipated energy. In this classification, the location of the stars, their detailed composition, and dimensions are ignored.

In engineering, a model may exist prior to the phenomena it represents. For example, a UML model may describe a system that has not yet been implemented. In science, the model may state the existence of systems and lead to the experiments that show their existence. For example, the theory behind the top quark was developed before accelerator experiments in CERN demonstrated the existence of the top quark.

In summary, modeling is the activity that software engineers perform when they design a system. The purpose of modeling is to construct an abstraction of the system that leaves out irrelevant details. Software engineers abstract concepts from the application domain (i.e., the environment in which the system is operating) and from the solution domain (i.e., the technologies for building a system). The resulting model is simpler than the environment or the system and thus is easier to manipulate. During the development of the model or its validation, software engineers need to communicate about the system with other engineers, clients, or users. They can represent the model in their imagination, on a napkin, with a CASE tool, using different notations. In doing so, they construct views of the model for supporting their specific communication need.

2.3.3 Data types, abstract data types, and instances

A **data type** is an abstraction in the context of a programming language. A data type has a unique name that distinguishes it from other data types, it has a purpose (i.e., the structure and the operations valid on all members of the data type), and it has members (i.e., the members of the data type). Data types are used in typed languages to ensure that only valid operations are applied to specific data members.

For example, the name `int` in Java corresponds to all the signed integers between -2^{32} and $2^{32} - 1$. The valid operations on the members of this type are all the integer arithmetic operations (e.g., addition, subtraction, multiplication, division) and all the functions and methods that have parameters of type `int` (e.g., `mod`). The Java run-time environment throws an exception if a floating point operation is applied to a member of the `int` data type (e.g., `trunc` or `floor`).

An **abstract data type** is a special data type whose structure is hidden from the rest of the system. This allows the developer to revise the internal structure and the implementation of the abstract data type without impacting the rest of the system.

For example, the abstract data type `Person` may define the operations `getName()`², `getSocialSecurityNumber()` and `getAddress()`. The fact that the social security number of the person is stored as a number or as a string is not visible to the rest of the system. Such decisions are called **implementation decisions**.

An **instance** is any member of a specific data type. For example, 1291 is an instance of the type `int`, and 3.14 is an instance of the type `float`. An instance of a data type can be manipulated with the operations defined by the data type.

The relationship between data type and instance is similar to the relationship between concept and phenomenon: A data type is an abstraction that describes a set of instances that share common characteristics. For example, the operation for renaming an instance of `Person` need only be defined once in the `Person` data type but will be applicable to all instances of `Person`.

2.3.4 Classes, abstract classes, and objects

A **class** is an abstraction in object-oriented programming languages. Like abstract data types, a class encapsulates both structure and behavior. Unlike abstract data types, classes can be defined in terms of other classes by using generalization. Assume we have a watch that also can function as a calculator. The class `CalculatorWatch` can then be seen as a refinement of the class `Watch`. This type of relationship between a base class and a refined class is called **generalization**. The base class (e.g., `Watch`) is called the **superclass**, the refined class is called the **subclass** (e.g., `CalculatorWatch`). In a generalization relationship, the subclass refines the superclass by defining new attributes and operations. In Figure 2-10, `CalculatorWatch` defines functionality for performing simple arithmetic that regular `Watches` do not have.

2. We refer to an operation by its name followed by its arguments in parentheses. If the arguments are not specified, we suffix the name of the operation by a pair of empty parentheses. We describe operations in detail in the next section.

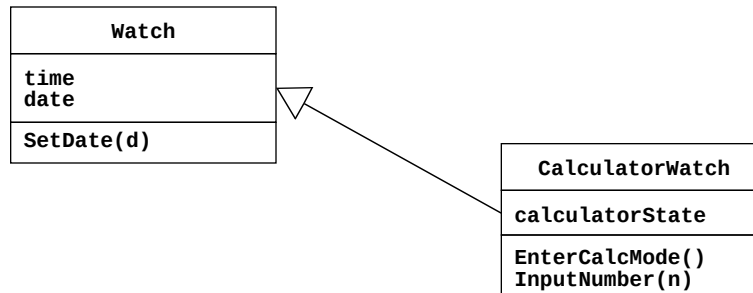


Figure 2-10 A UML class diagram depicting two classes: *Watch* and *CalculatorWatch*. *CalculatorWatch* is a refinement of *Watch*, providing calculator functionality normally not found in normal watches. In a UML class diagram, classes and objects are represented as boxes with three compartments: The first compartment depicts the name of the class, the second depicts its attributes, the third its operations. The second and third compartments can be omitted for brevity. An inheritance relationship is displayed by a line terminated with a triangle. The triangle points to the superclass, and the other end is attached to the subclass.

When a generalization serves only the purpose of modeling shared attributes and operations, that is, if the generalization is never instantiated, it is called an **abstract class**. Abstract classes often represent generalized concepts in the application domain. Whenever we classify phenomena into concepts, they often create generalizations to manage the complexity of the classification. For example, in chemistry, *Benzenecan* can be considered a class of molecules that belongs to the abstract class *OrganicCompound* (Figure 2-11). Note that *OrganicCompound* is a generalization and does not correspond to any one molecule, that is, it does not have any instances. When modeling software systems, abstract classes sometimes do not correspond to an existing application domain concept, but rather are introduced to reduce complexity in the model or to promote reuse.

A class defines the **operations** that can be applied to its instances. Operations of a superclass can be inherited and applied to the objects of the subclass as well. For example, in Figure 2-10, the operation *SetDate(d)* setting the current date of a *Watch* is also applicable for *CalculatorWatch*s. The operation *EnterCalcMode()* however, defined in the *CalculatorWatch* class, is not applicable in the *Watch* class.

A class defines the **attributes** that apply to all its instances. An attribute is a named slot in the instance where a value is stored. Attributes have a unique name within the class and a type. *Watches* have a *time* and a *date* attribute. *CalculatorWatches* have a *calculatorState* attribute.

An **object** is an instance of a class. An object has an identity and stores attribute values. Each object belongs to exactly one class. In UML, an instance is depicted by a rectangle with its name underlined. This convention is used throughout UML to distinguish between instances and types.³ In Figure 2-12, simpleWatch1291 is an instance of *Watch* and calculatorWatch1515

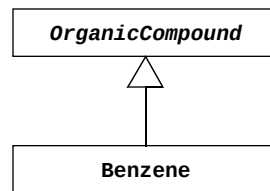


Figure 2-11 An example of abstract class (UML class diagram). *Organic Compound* is never instantiated and only serves as a generalization.

is an instance of CalculatorWatchNote that, although the operations of Watch are applicable to calculatorWatch1515, calculatorWatch1515 is not an instance of the class Watch. Unlike abstract data types, the attributes of an object can be visible to other parts of the system in some programming languages. For example, Java allows the implementer to specify in great detail which attributes are visible and which are not.

2.3.5 Event classes, events, and messages

Event classes are abstractions representing a kind of events for which the system has a common response. An **event**, an instance of an event class, is a relevant occurrence in the system. For example, an event can be a stimuli from an actor (e.g., “the WatchUser presses the left button”), a time-out (e.g., “after 2 minutes”), or the sending of a message between two objects. Sending a **message** is the mechanism by which the sending object requests the execution of an operation in the receiving object. The message is composed of a name and a

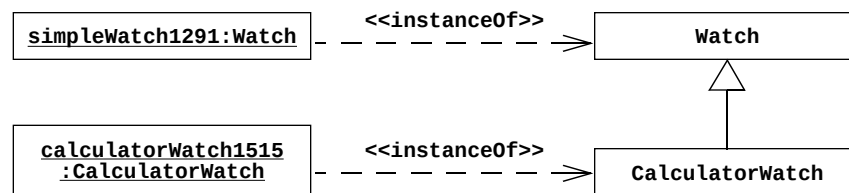


Figure 2-12 A UML class diagram depicting instances of two classes. simpleWatch1291 is an instance of Watch. calculatorWatch1515 is an instance of CalculatorWatch. Although the operations of Watch are also applicable to calculatorWatch1515, the latter is not an instance of the former.

3. Underlined strings are also used for representing Uniform Resource Locators (URLs). To improve readability, we do not use an underlined font in the text, but rather, use the same font to denote instances and classes. In general, this ambiguity can be resolved by examining the context. In UML diagrams, however, we always use an underlined font to distinguish instances from classes.

number of arguments. The receiving object matches the name of the message to one of its operation and passes the arguments to the operation. Results are returned to the sending object.

For example, in Figure 2-13, the Watch object computes the current time by getting the Greenwich time from the Time object and the time difference from the TimeZone object by sending the `getTime()` and the `getTimeDelta()` messages, respectively.

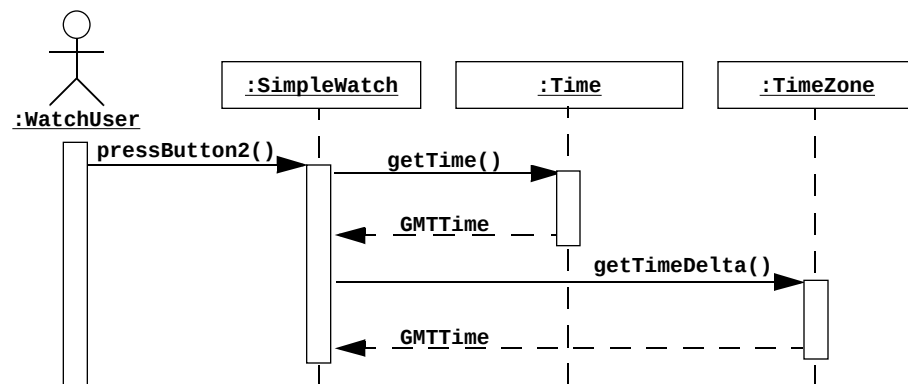


Figure 2-13 Examples of message sends (UML sequence diagram). The Watch object sends the `getTime()` message to a Time object to query the current Greenwich time. It then sends the `getTimeDelta()` message to a TimeZone object to query the difference to add to the Greenwich time. The dashed arrows represent the results that are sent back to the message sender.

Events and messages are instances: They represent concrete occurrences in the system. Event classes are abstractions describing groups of events for which the system has a common response. In practice, the term “event” can refer to instances or classes. This ambiguity is resolved by examining the context in which the term is used.

2.3.6 Object-oriented modeling

The **application domain** represents all aspects of the user’s problem. This includes the physical environment, the users and other people, their work processes, and so on. It is critical for analysts and developers to understand the application domain for a system to accomplish its intended task effectively. Note that the application domain changes over time, as work processes and people change.⁴

4. The application domain is sometimes further divided into a user domain and a client domain. The client domain includes the issues relevant to the client, such as, operation cost of the system, impact of the system on the rest of the organization. The user domain includes the issues relevant to the end user, such as, functionality, ease of learning and of use.

The **solution domain** is the space of all possible systems. The solution domain is much richer and more volatile than the application domain. This is due to emerging technologies (also called technology enablers), to changes as the implementation technology matures, or better understanding of implementation technology by the developers when they build the system. Modeling the solution domain represents the system design and object design activities of the development process. Note that the deployment of the system can change the application domain, as users develop new work processes to accommodate the system.

Object-oriented analysis is concerned with modeling the application domain. **Object-oriented design** is concerned with modeling the solution domain. Both modeling activities use the same representations (i.e., classes and objects). In object-oriented analysis and design, the application domain model is also part of the system model. For example, an air traffic control system has a `TrafficController` class to represent individual users, their preferences, and log information. The system also has an `Aircraft` class to represent information associated with the tracked aircraft. `TrafficController` and `Aircraft` are application domain concepts that are encoded into the system (Figure 2-14).

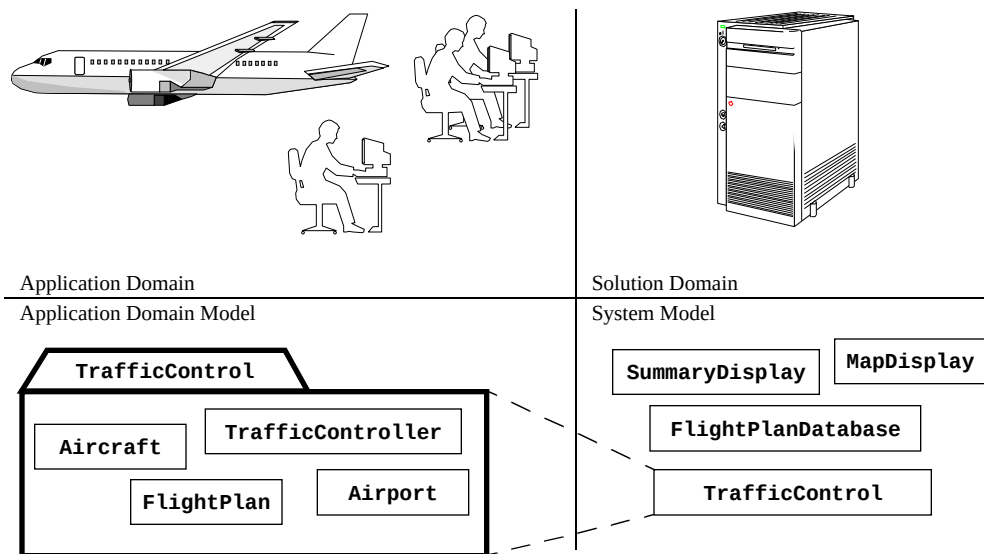


Figure 2-14 The application domain model represents entities of the environment which are relevant to an air traffic control system (e.g., aircraft, traffic controllers). The system model represents entities that are part of the system (e.g., map display, flight plan database). In object-oriented analysis and design, the application domain model is also part of the system model. An example in this figure is the `TrafficControl` package that appears in both models. (For more details, see Chapter 5, *Analysis*).

Modeling the application domain and the solution domain with a single notation has advantages and disadvantages. On the one hand, it can be powerful: Solution domain classes that represent application concepts can be traced back to the application domain. Moreover, these classes can be encapsulated into subsystems independent of other implementation concepts (e.g., user interface and database technology) and be packaged into a reusable toolkit of domain classes. On the other hand, using a single notation can introduce confusion because it removes the distinction between the real world and the model of it. The system domain is bound to be simpler and biased toward the solution. To address this issue, we use a single notation and, in cases of ambiguity, we distinguish between the two domains. In most cases, we are referring to the model (e.g., “an Aircraft is composed of Manifest and a FlightPlan” is a statement about the model).

2.3.7 Falsification and prototyping

A model is a simplification of reality in the sense that irrelevant details are ignored. Relevant details, however, need to be represented. **Falsification** [Popper, 1992] is the process of demonstrating that relevant details have been incorrectly represented or not represented at all; that is, the model does not correspond to the reality it is supposed to represent.

The process of falsification is well known in other sciences: Researchers propose different models of a reality, which are gradually accepted as an increasing amount of data supports the model, but which are rejected once a counterexample is found. Ptolemy’s earth-centric model of the universe was (eventually) falsified in favor of the Copernican solar-centric model once Galileo’s data were accepted. The Copernican solar-centric model was then falsified once other galaxies were discovered and the concept of galaxy was added to the model.

We can apply falsification to software system development as well. For example, a technique for developing a system is **prototyping**: When designing the user interface, developers construct a prototype that only simulates the user interface of a system. The prototype is then presented to potential users for evaluation, that is, falsification, and modified subsequently. In the first iterations of this process, developers are likely to throw away the initial prototype as a result of feedback from the users. In other terms, users falsify the initial prototype, a model of the future system, because it does not represent accurately relevant details.

Note that it is only possible to demonstrate that a model is incorrect. Although, in some cases, it is possible to show mathematically that two models are equivalent, it is not possible to show that either of them correctly represents reality. For example, formal verification techniques can enable developers to show that a specific software implementation is consistent with a formal specification. However, only field testing and extended use can indicate that a system meets the needs of the client. At any time, system models can be falsified due to changes in the requirements, in the implementation technology, or in the environment.

2.4 A deeper view into UML

We now describe in detail the five main UML diagrams we use in this book.

- **Use case diagrams** represent the functionality of the system from a user's point of view. They define the boundaries of the system (Section 2.4.1).
- **Class diagrams** are used to represent the structure of a system in terms of objects, their attributes, and relationships (Section 2.4.2).
- **Sequence diagrams** represent the system's behavior in terms of interactions among a set of objects. They are used to identify objects in the application and implementation domains (Section 2.4.3).
- **Statechart diagrams** are used to represent the behavior of nontrivial objects (Section 2.4.4).
- **Activity diagrams** are flow diagrams used to represent the data flow or the control flow through a system (Section 2.4.5).

2.4.1 Use case diagrams

Use cases and actors

Actors are external entities that interact with the system. Examples of actors include a user role (e.g., a system administrator, a bank customer, a bank teller) or another system (e.g., a central database, a fabrication line). Actors have unique names and descriptions.

Use cases describe the behavior of the system as seen from an actor's point of view. Behavior described by the use case model is also called **external behavior**. A use case describes a function provided by the system as a set of events that yields a visible result for the actors. Actors initiate a use case to access the system functionality. The use case can then initiate other use cases and gather more information from the actors. When actors and use cases exchange information, they are said to **communicate**. We will see later that we represent these exchanges with communication relationships.

For example, in an accident management system [FRIEND, 1994], field officers, such as, a police officer or a fireman, have access to a wireless computer that enables them to interact with a dispatcher. The dispatcher in turn can visualize the current status of all its resources, such as, police cars or trucks, on a computer screen and dispatch a resource by issuing commands from a workstation. In this example, `FieldOfficer` and `Dispatcher` are actors.

Figure 2-15 depicts the actor `FieldOfficer` who invokes the use case `ReportEmergency` to notify the actor `Dispatcher` of a new emergency. As a response, the `Dispatcher` invokes the `OpenIncident` use case to create an incident report and initiate the incident handling. The `Dispatcher` enters preliminary information from the `FieldOfficer` in the incident database and orders additional units to the scene with the `AllocateResources` use case.

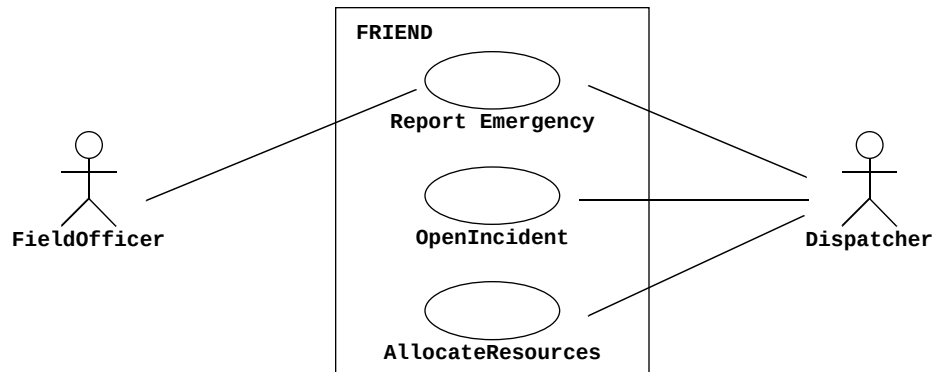


Figure 2-15 An example of a UML use case diagram: Incident initiation in an accident management system. Associations between actors and use cases represent information flows. In UML, these associations are bidirectional: They can represent the actor initiating a use case (e.g., *FieldOfficer* initiates *ReportEmergency*) or a use case providing information to an actor (e.g., *ReportEmergency* notifies *Dispatcher*).

To describe a use case, we use a template composed of six fields (see also Figure 2-16):

- The **name** of the use case is unique across the system so that developers (and project participants) can unambiguously refer to the use case.
- **Participating actors** are actors interacting with the use case.
- **Entry conditions** describe the conditions that need to be satisfied before the use case is initiated.
- The **flow of events** describes the sequence of actions of the use case, which are be numbered for reference. The common case (i.e., cases that occur frequently) and the exceptional cases (i.e., cases that seldom occur, such as errors and unusual conditions) are described separately in different use cases for clarity.
- **Exit conditions** describe the conditions that are satisfied after the completion of the use case.
- **Special requirements** are requirements that are not related to the functionality of the system. These include constraints on the performance of the system, its implementation, the hardware platforms it runs on, and so on. Special requirements are described in detail in Chapter 4, *Requirements Elicitation*.

Use cases are written in natural language. This enables developers to use them for communicating with the client and the users, who generally do not have an extensive knowledge of software engineering notations. The use of natural language also enables participants from other disciplines to understand the requirements of the system.

<i>Use case name</i>	ReportEmergency
<i>Participating actor</i>	Invoked by FieldOfficer Communicates with Dispatcher
<i>Entry condition</i>	1. The FieldOfficer activates the “Report Emergency” function of her terminal. FRIEND responds by presenting a form to the officer.
<i>Flow of events</i>	2. The FieldOfficer fills the form by selecting the emergency level, type, location, and brief description of the situation. The FieldOfficer also describes possible responses to the emergency situation. Once the form is completed, the FieldOfficer submits the form, at which point the Dispatcher is notified. 3. The Dispatcher reviews the submitted information and creates an Incident in the database by invoking the OpenIncident use case. The Dispatcher selects a response and acknowledges the emergency report.
<i>Exit condition</i>	4. The FieldOfficer receives the acknowledgment and the selected response.
<i>Special requirements</i>	The FieldOfficer’s report is acknowledged within 30 seconds. The selected response arrives no later than 30 seconds after it is sent by the Dispatcher.

Figure 2-16 An example of a use case: the ReportEmergency use case.

Scenarios

A use case is an abstraction that describes all possible scenarios involving the described functionality. A **scenario** is an instance of a use case describing a concrete set of actions. Scenarios are used as examples for illustrating common cases. Their focus is on understandability. Use cases are used to describe all possible cases. Their focus is on completeness. We describe a scenario using a template with three fields:

- The **name** of the scenario enables us to refer to it unambiguously. The name of a scenario is underlined to indicate that it is an instance.
- The **participating actor instances** field indicates which actor instances are involved in this scenario. Actor instances also have underlined names.
- The **flow of events** of a scenario describes the sequence of events step by step.

Note that, there is no need for entry or exit conditions in scenarios. Entry and exit conditions are abstractions that enable developers to describe a range of conditions under which a use case is invoked. Given that a scenario only describes a single flow of events, such conditions are unnecessary (Figure 2-17).

Use case diagrams can include four types of relationships: communication, inclusion, extension, and generalization. We describe these relationships in detail next.

<i>Scenario name</i>	<u>warehouseOnFire</u>
<i>Participating actor instances</i>	<u>bob, alice: FieldOfficer</u> <u>john: Dispatcher</u>
<i>Flow of events</i>	1. Bob, driving down main street in his patrol car, notices smoke coming out of a warehouse. His partner, Alice, activates the “Report Emergency” function from her FRIEND laptop. 2. Alice enters the address of the building, a brief description of its location (i.e., northwest corner), and an emergency level. In addition to a fire unit, she requests several paramedic units on the scene given that area appears to be relatively busy. She confirms her input and waits for an acknowledgment. 3. John, the Dispatcher, is alerted to the emergency by a beep of his workstation. He reviews the information submitted by Alice and acknowledges the report. He allocates a fire unit and two paramedic units to the Incident site and sends their estimated arrival time (ETA) to Alice. 4. Alice receives the acknowledgment and the ETA.

Figure 2-17 The warehouseOnFire scenario for the ReportEmergency use case.

Communication relationships

Actors and use cases **communicate** when information is exchanged between them. Communication relationships are depicted by a solid line between the actor and use case symbol. In Figure 2-15, the actors `FieldOfficer` and `Dispatcher` communicate with the `ReportEmergency` use case. Only the actor `Dispatcher` communicates with the use cases `OpenIncident` and `AllocateResources`. Communication relationships between actors and use cases can be used to denote access to functionality. In the case of our example, a `FieldOfficer` and a `Dispatcher` are provided with different interfaces to the system and have access to different functionalities.

Include relationships

When describing a complex system, its use case model can become quite complex and can contain redundancy. We reduce the complexity of the model by identifying commonalities in different use cases. For example, assume that the `Dispatcher` can press at any time a key to access Help. This can be modeled by a use case `HelpDispatcher` that is included by the use cases `OpenIncident` and `AllocateResources` (and any other use cases accessible by the `Dispatcher`). The resulting model only describes the `HelpDispatcher` functionality once, thus reducing complexity. Two use cases are related by an include relationship if one of them includes the second one in its flow of events. In UML, **include relationships** are depicted by a dashed arrow originating from the including use case (see Figure 2-18). Include relationships are labeled with the string `<<include>>`.

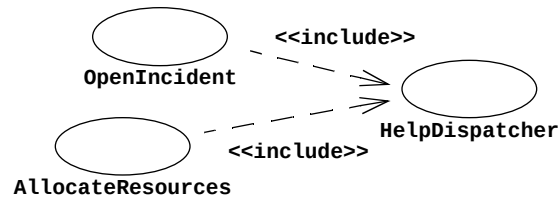


Figure 2-18 An example of an `<<include>>` relationship (UML use case diagram).

We represent include relationships in the use case itself with one of two ways. If the included use case can be included at any point in the flow of events (e.g., the *HelpDispatcher* use case), we indicate the inclusion in the *Special requirements* section of the use case. If the included use case is explicitly invoked during an event, we indicate the inclusion in the flow of events.

Extend relationships

Extend relationships are an alternate means for reducing complexity in the use case model. A use case can extend another use case by adding events. An extend relationship indicates that an instance of an extended use case may include (under certain conditions) the behavior specified by the extending use case. A typical application of extend relationships is the specification of exceptional behavior. For example (Figure 2-19), assume that the network connection between the *Dispatcher* and the *FieldOfficer* can be interrupted at any time. (e.g., if the *FieldOfficer* enters a tunnel). The use case *ConnectionDown* describes the set of events taken by the system and the actors while the connection is lost. *ConnectionDown* extends the use cases *OpenIncident* and *AllocateResources*. Separating exceptional behavior from common behavior enables us to write shorter and more focused use cases.

In the textual representation of a use case, we represent extend relationships as entry conditions of the extending use case. For example, the extend relationships depicted in Figure 2-19 are represented as an entry condition of the *ConnectionDown* use case (Figure 2-20).

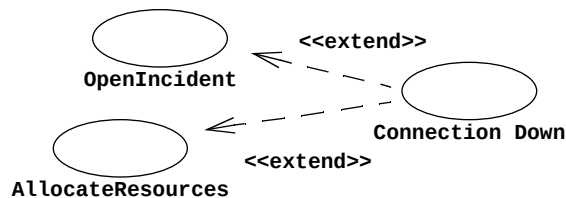


Figure 2-19 An example of an `<<extend>>` relationship (UML use case diagram).

<i>Use case name</i>	ConnectionDown
<i>Participating actor</i>	Communicates with FieldOfficer and Dispatcher.
<i>Entry condition</i>	This use case extends the OpenIncident and the AllocateResources use cases. It is initiated by the system whenever the network connection between the FieldOfficer and Dispatcher is lost.
<i>Flow of events</i>	1. ...

Figure 2-20 Textual representation of extend relationships of Figure 2-19.

The difference between the include and extend relationships is the location of the dependency. Assume that we add several new use cases for the actor Dispatcher. If we modeled the HelpDispatcherfunction with include relationships, every new use case will need to include the HelpDispatcheruse case. If we used extend relationships instead, only the HelpDispatcheruse case needs to be modified to extend the additional use cases. In general, exception cases, such as help, errors, and other unexpected conditions, are modeled with extend relationships. Use cases that describe behavior commonly shared by a fixed set of use cases are modeled with include relationships.

Generalization relationships

Generalization/specialization relationships are a third mechanism for reducing the complexity of a model. A use case can specialize a more general one by adding more detail. For example, FieldOfficers are required to authenticate before they can use FRIEND. During early stages of requirements elicitation, authentication is modeled as a high-level Authenticateuse case. Later, developers describe the Authenticateuse case in more detail and allow for several different hardware platforms. This refinement activity results in two more use cases, AuthenticateWithPassword, which enables FieldOfficers to login without any specific hardware, and AuthenticateWithCard, which enables FieldOfficers to login using a smart card. The two new use cases are represented as specializations of the Authenticateuse case (Figure 2-21).

Applying use case diagrams

Use cases and actors define the boundaries of the system. They are developed during requirements elicitation, often with the client and the users. During analysis, use cases are refined and corrected as they are reviewed by a broader audience that includes developers and validated against real situations.

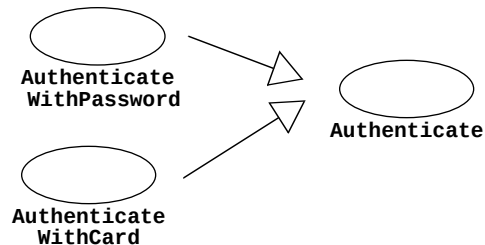


Figure 2-21 An example of a generalization relationship (UML use case diagram). The *Authenticate* use case is a high-level use case describing, in general terms, the process of authentication. *AuthenticateWithPassword* and *AuthenticateWithCard* are two specializations of *Authenticate*.

2.4.2 Class diagrams

Classes and objects

Class diagrams describe the structure of the system in terms of classes and objects. **Classes** are abstractions that specify the attributes and behavior of a set of objects. **Objects** are entities that encapsulate state and behavior. Each object has an identity: It can be referred individually and is distinguishable from other objects.

In UML, classes and objects are depicted by boxes including three compartments. The top compartment displays the name of the class or object. The center compartment displays its attributes, and the bottom compartment displays its operations. The attribute and operation compartment can be omitted for clarity. Object names are underlined to indicate that they are instances. By convention, class names start with an uppercase letter. Objects in object diagrams may be given names (followed by their class) for ease of reference. In that case, their name starts with a lowercase letter. In the FRIEND example (Figures 2-22 and 2-23), Bob and Alice are field officers, represented in the system as *FieldOfficer* objects called *bob:FieldOfficer* and *alice:FieldOfficer*. *FieldOfficer* is a class describing all *FieldOfficer* objects, whereas Bob and Alice are represented by two individual *FieldOfficer* objects.

In Figure 2-22, the *FieldOfficer* class has two attributes: a name and a badgeNumber. This indicates that all *FieldOfficer* objects have these two attributes. In Figure 2-23, the *bob:FieldOfficer* and *alice:FieldOfficer* objects have specific values for these attributes: "Bob. D." and "Alice W.", respectively. In Figure 2-22, the *FieldOfficer.name* attribute is of type *String* which indicates that only instances of *String* can be assigned to the *FieldOfficer.name* attribute. The type of an attribute is used to specify the valid range of values the attribute can take. Note that when attribute types are not essential to the definition of the system, attribute type decisions can be delayed until object design. This allows the developers to concentrate on the functionality of the system and to minimize the number of trivial changes when the functionality of the system is revised.

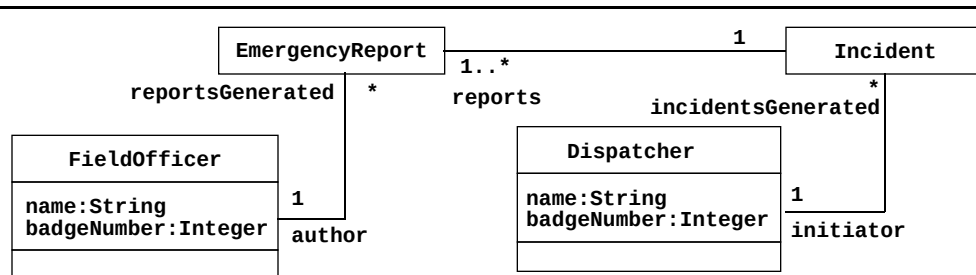


Figure 2-22 An example of a UML class diagram: classes that participate in the ReportEmergency use case. Detailed type information is usually omitted until object design (see Chapter 7, *Object Design*).

Associations and links

A **link** represents a connection between two objects. **Associations** are relationships between classes and represent groups of links. Each **FieldOfficer** object also has a list of **EmergencyReport** objects that has been written by the **FieldOfficer**. In Figure 2-22, the line between the **FieldOfficer** class and the **EmergencyReport** class is an association. In Figure 2-23, the line between the **alice:FieldOfficer** object and the **report_1291:EmergencyReport** object is a link. This link represents state that is kept in the system to denote that **alice:FieldOfficer** generated **report_1291:EmergencyReport**.

Roles

Each end of an association can be labeled by a string called **role**. In Figure 2-22, the roles of the association between the **EmergencyReport** and **FieldOfficer** classes are **author** and **reportsGenerated**. Labeling the end of associations with roles allows us to distinguish multiple associations originating from a class. Moreover, roles clarify the purpose of the association.

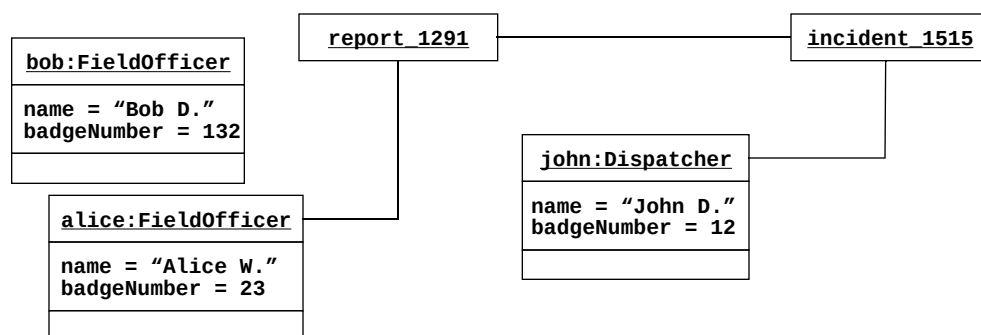


Figure 2-23 An example of a UML object diagram: objects that participate in the warehouseOnFire scenario.

Multiplicity

Each end of an association can be labeled by a set of integers indicating the number of links that can legitimately originate from an instance of the class connected to the association end. The association end `author` has a multiplicity of 1. This means that all `EmergencyReport`s are written by exactly one `FieldOfficer`. In other terms, each `EmergencyReport` object has exactly one link to an object of class `FieldOfficer`. The multiplicity of the association end `reportsGenerated` is “many,” shown as a star. The “many” multiplicity is shorthand for 0..n. This means that any given `FieldOfficer` may be the author of zero or more `EmergencyReport`s.

Association class

Associations are similar to classes, in that they can have attributes and operations attached to them. Such an association is called an **association class** and is depicted by a class symbol, containing the attributes and operations, connected to the association symbol with a dashed line. For example, in Figure 2-24, the allocation of `FieldOfficer`s to an `Incident` is modeled as an association class with attributes `role` and `notificationTime`.

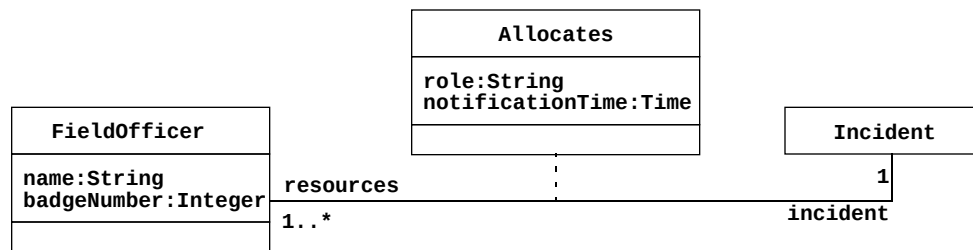


Figure 2-24 An example of an association class (UML class diagram).

Any association class can be transformed into a class and simple associations as shown in Figure 2-25. Although both representations are similar, the association class representation is clearer: An association cannot exist without the classes it links. Similarly, the `Allocation` object cannot exist without a `FieldOfficer` and an `Incident` object. Although Figure 2-25 carries the same information, this diagram requires careful examination of the association multiplicity. We examine such modeling trade-offs in Chapter 5, *Analysis*.

Aggregation

Associations are used to represent a wide range of connections among a set of objects. In practice, a special case of association occurs frequently: composition. For example, a `State` contains many `Counties`, which in turn contains many `Townships`. A `PoliceStation` is composed of `PoliceOfficers`. Another example is a `Directory` that contains a number of `Files`. Such relationships could be modeled using a one-to-many association. Instead, UML provides the concept of an **aggregation** to denote composition. An aggregation is denoted by a

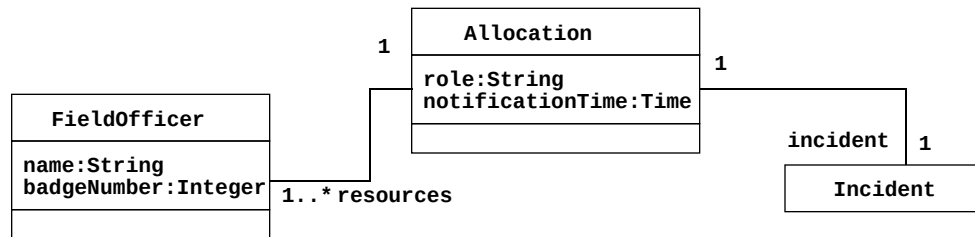


Figure 2-25 Alternative model for Allocation (UML class diagram).

simple line with a diamond at the container end of the association (see Figure 2-26). Although one-to-many associations and aggregations can be used alternatively, aggregations are preferable because they emphasize the hierarchical aspects of the relationship. For example, in Figure 2-26, the *PoliceOfficers* are part of the *PoliceStation*

Generalization

Generalization is the relationship between a general class and one or more specialized classes. Generalization enables us to describe all the attributes and operations that are common to a set of classes. For example, *FieldOfficer* and *Dispatcher* both have name and badgeNumber attributes. However, *FieldOfficer* has an association with *EmergencyReport*, whereas *Dispatcher* has an association with *Incident*. The common attributes of *FieldOfficer* and *Dispatcher* can be modeled by introducing a *PoliceOfficer* class that is specialized by the *FieldOfficer* and the *Dispatcher* classes (see Figure 2-27). *PoliceOfficer*, the generalization, is called a **superclass**. *FieldOfficer* and *Dispatcher*, the specializations, are called the **subclasses**. The subclasses **inherit** the attributes and operations from their parent class. Abstract classes (defined in Section 2.3.4) are distinguished from concrete classes by *italicizing* the name of abstract classes. In Figure 2-27, *PoliceOfficer* is an abstract class. Abstract classes are used in object-oriented modeling to classify related concepts, thus reducing the overall complexity of the model.

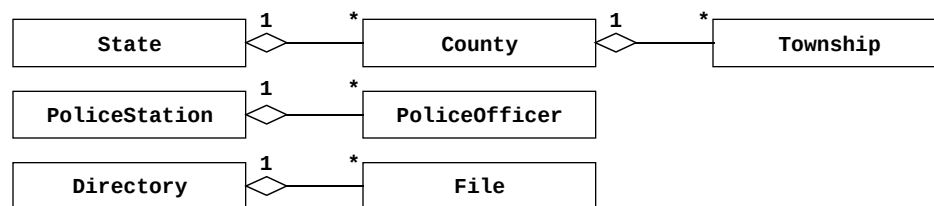


Figure 2-26 Examples of aggregations (UML class diagram). A State contains many Counties, which in turn contains many Townships. A PoliceStation has many PoliceOfficers. A file system Directory contains many Files.

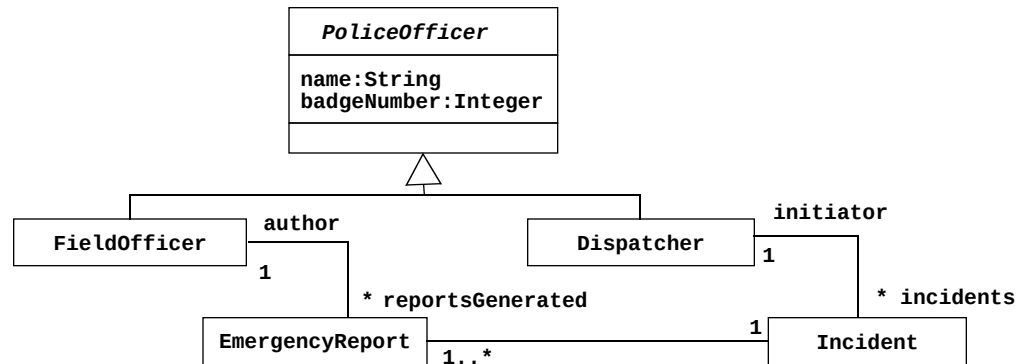


Figure 2-27 An example of a generalization (UML class diagram). *PoliceOfficer* is an abstract class which defines the common attributes and operations of the *FieldOfficer* and *Dispatcher* classes.

Object behavior is specified by **operations**. A set of operations represents a **service** offered by a particular class. An object requests the execution of an operation from another object by sending it a **message**. The message is matched up with a **method** defined by the class to which the receiving object belongs or by any of its superclasses. The operations of a class are the public services that the class offers. The methods of its class are the implementations of these operations.

The distinction between operations and methods allows for a cleaner separation between the mechanism to request a service and the location where it is provided. For example, the class *Incident* in Figure 2-28 defines an operation, called *assignResource()*, which, given a *FieldOfficer*, creates an association between the receiving *Incident* and the specified *Resource*. The *assignResource()* operation may also have a side effect such as sending a notification to the newly assigned *Resource*. The *close()* operation of *Incident* is responsible for closing the *Incident*. This includes going over all the resources that have been assigned to the incident over time and collecting their reports.

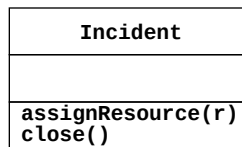


Figure 2-28 Examples of operations provided by the *Incident* class (UML class diagram).

Applying class diagrams

Class diagrams are used for describing the structure of a system. During analysis, software engineers build class diagrams to formalize application domain knowledge. Classes represent participating objects found in use cases and sequence diagrams and describe their attributes and operations. The purpose of analysis models is to describe the scope of the system and discover its boundaries. For example, using the class diagram pictured in Figure 2-22, an analyst could examine the multiplicity of the association between `FieldOfficer` and `EmergencyReport` (i.e., one `FieldOfficer` can write zero or more `EmergencyReport`s, but each `EmergencyReport` is written by exactly one `FieldOfficer`) and ask the user whether this is correct. Can there be more than one author given an `EmergencyReport`? Can there be anonymous reports? Depending on the answer from the user, the analyst would then change the model to reflect the application domain. The development of analysis models is described in Chapter 5, *Analysis*.

Analysis models do not focus on implementation. Concepts such as interface details, network communication, and database storage are not represented. Class diagrams are refined during system design and object design to include classes representing the solution domain. For example, the developer adds classes representing databases, user interface windows, adapters around legacy code, optimizations, and so on. The classes are also grouped into subsystems with well-defined interfaces. The development of design models is described in Chapter 6, *System Design*, and Chapter 7, *Object Design*.

2.4.3 Sequence diagrams

Sequence diagrams describe patterns of communication among a set of interacting objects. An object interacts with another object by sending **messages**. The reception of a message by an object triggers the execution of an operation, which in turn may send messages to other objects. **Arguments** may be passed along with a message and are bound to the parameters of the executing operation in the receiving object.

For example, consider a watch with two buttons (hereafter called 2Bwatch). Setting the time on 2Bwatch requires the actor 2BwatchOwner to first press both buttons simultaneously, after which 2Bwatch enters the set time mode. In the set time mode, 2Bwatch blinks the number being changed (e.g., the hours, the minutes, or the seconds, day, month, year). Initially, when the 2BwatchOwner enters the set time mode, the hours blink. If the actor presses the first button, the next number blinks (e.g., if the hours are blinking and the actor presses the first button, the hours stop blinking and the minutes start blinking. If the actor presses the second button, the blinking number is incremented by one unit. If the blinking number reaches the end of its range, it is reset to the beginning of its range (e.g., assume the minutes are blinking and its current value is 59, its new value is set to 0 if the actor presses the second button). The actor exits the set time mode by pressing both buttons simultaneously. Figure 2-29 depicts a sequence diagram for an actor setting his 2Bwatch one minute ahead.

Each column represents an object that participates in the interaction. The vertical axis represents time from top to bottom. Messages are shown by arrows. Labels on arrows represent

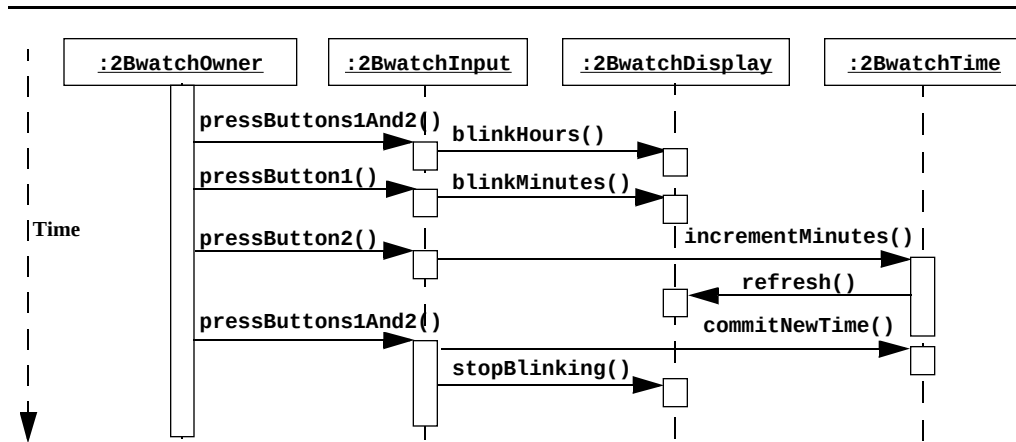


Figure 2-29 Example of a sequence diagram: setting the time on 2Bwatch.

message names and may contain arguments. Activations (i.e., executing methods) are depicted by vertical rectangles. Actors are shown as the leftmost column.

Sequence diagrams can be used to describe either an abstract sequence (i.e., all possible interactions) or concrete sequences (i.e., one possible interaction, as in Figure 2-29). When describing all possible interactions, sequence diagrams also provide notations for conditionals and iterators. A condition on a message is denoted by an expression in brackets before the message name (see `[i>0] op1()` and `[i<=0] op2()` in Figure 2-30). If the expression is True, the message is sent. Repetitive invocation of a message is denoted by a “*” before the message name (see `*op3()` in Figure 2-30).

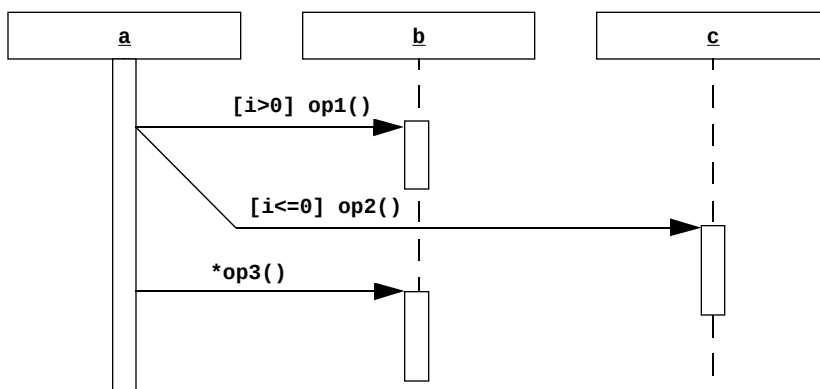


Figure 2-30 Examples of conditions and iterators in sequence diagrams.

Applying sequence diagrams

Sequence diagrams describe interactions among several objects. Typically, we use a sequence diagram to describe the event flow of a use case, identify the objects that participate in the use case, and assign pieces of the use case behavior to the objects in the form of services. This process often leads to refinements in the use case (e.g., correcting ambiguous descriptions, adding missing behavior) and consequently, the discovery of more objects and more services. We describe in detail the use of sequence diagrams in Chapter 5, *Analysis*.

2.4.4 Statechart diagrams

A **UML statechart** is a notation for describing the sequence of states an object goes through in response to external events. Statecharts are extensions of the finite state machine model. On the one hand, statecharts provide notation for nesting states and state machines (i.e., a state can be described by a state machine). On the other hand, statecharts provide notation for binding transitions with message sends and conditions on objects. UML statecharts are based on Harel's statecharts [Harel, 1987]. A UML statechart is equivalent to a Mealy or Moore state machine.

A **state** is a condition that an object satisfies. A state can be thought of as an abstraction of the attribute values of a class. For example, an Incident object in FRIEND can be in four states: *Active*, *Inactive*, *Closed*, and *Archived* (see Figure 2-31). An active Incident denotes a situation that requires a response (e.g., an ongoing fire, a traffic accident). An inactive Incident denotes a situation that was handled but for which reports still need to be written (e.g., the fire has been put out but damage estimates have not yet been performed). A closed Incident denotes a situation that has been handled and documented. An archived Incident is a closed Incident whose documentation has been moved to off-site storage.

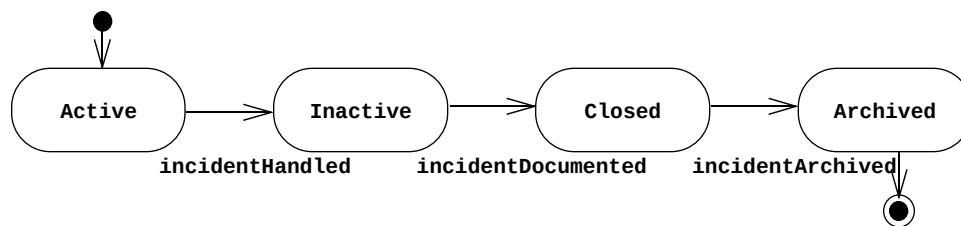


Figure 2-31 A UML statechart diagram for the Incident class.

A **transition** represents changes of state triggered by events, conditions, or time. For example, in Figure 2-31, there are three transitions: from the *Active* state into the *Inactive* state, from the *Inactive* state to the *Closed* state, and from the *Closed* state to the *Archived* state.

A state is depicted by a rounded rectangle. A transition is depicted by arrows connecting two states. States are labeled with their name. A small solid black circle indicates the initial state. A circle surrounding a small solid black circle indicates a final state.

Figure 2-32 displays another example, a statechart for the 2Bwatch (for which we constructed a sequence diagram in Figure 2-29). At the highest level of abstraction, 2Bwatch has two states, `MeasureTime` and `SetTime`. 2Bwatch changes states when the user presses and releases both buttons simultaneously. When 2Bwatch is first powered on, it is in the `SetTime` state. This is indicated by the small solid black circle, which represents the initial state. When the battery of the watch runs out, the 2Bwatch is permanently out of order. This is indicated with a final state. In this example, transitions can be triggered by an event (e.g., `pressButtonsLAndR`) or by the passage of time (e.g., `after 2 min.`). Actions can be associated with a transition (e.g., `beep` when the transition between `SetTime` and `MeasureTime` is fired on the `pressButtonsLAndR` event).

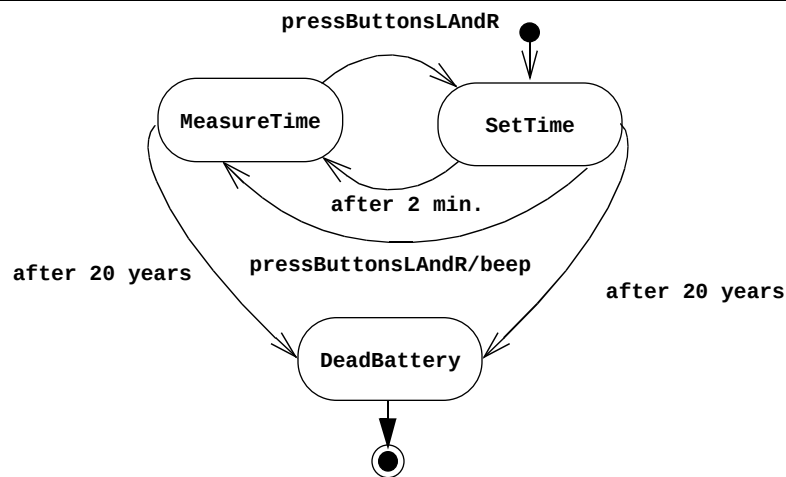


Figure 2-32 Statechart diagram for 2Bwatch set time function.

The statechart diagram in Figure 2-32 does not represent the details of measuring or setting the time. These details have been abstracted away from the statechart diagram and can be modeled separately using either internal transitions or a nested statechart. Internal transitions (Figure 2-33) are transitions that remain within a single state. They can also have actions associated with them. Entry and exit are displayed as an internal transition, given that their actions do not depend on the originating and destination states.

Nested statecharts reduce complexity. They can be used instead of internal transitions. In Figure 2-34, the current number is modeled as nested state, whereas actions corresponding to modifying the current number are modeled using internal transitions. Note that each state could

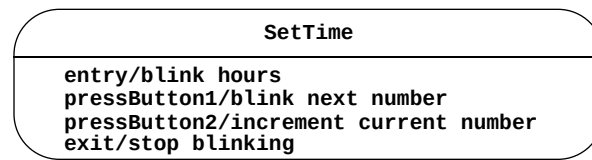


Figure 2-33 Internal transitions associated with the SetTime state (UML statechart diagram).

be modeled as a nested statechart (e.g., the **BlinkHour** statechart would have 24 substates that correspond to the hours in the day; transitions between these states would correspond to pressing the second button).

Applying statechart diagrams

Statechart diagrams are used to represent nontrivial behavior of a subsystem or an object. Unlike sequence diagrams that focus on the events impacting the behavior of a set of objects, statechart diagrams make explicit which attribute or set of attributes have an impact on the behavior of a single object. Statecharts are used to identify object attributes and to refine the behavior description of an object, and sequence diagrams are used to identify participating objects and the services they provide. Statechart diagrams can also be used during system and object design to describe solution domain objects with interesting behavior. We describe the use of statechart diagrams in detail in Chapter 5, *Analysis*, and Chapter 6, *System Design*.

2.4.5 Activity diagrams

The outgoing transitions are triggered by the completion of an action associated with the state. This is called an **action state**. By convention, the name of a state denotes a condition, whereas the name of an action state denotes an action. **Activity diagrams** are statechart

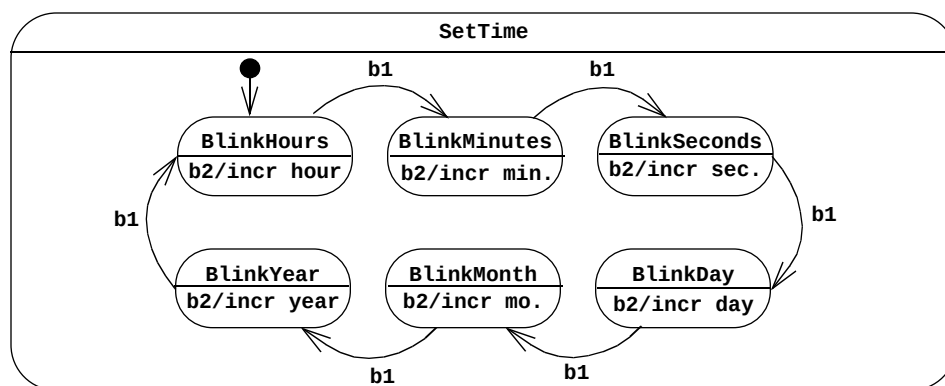


Figure 2-34 Refined statechart associated with the SetTime state (UML statechart diagram).



Figure 2-35 A UML activity diagram for Incident. During the action state HandleIncident, the Dispatcher receives reports and allocates resources. Once the Incident is closed, the Incident moves to the DocumentIncident activity during which all participating FieldOfficers and Dispatchers document the Incident. Finally, the ArchiveIncident activity represents the archival of the Incident related information onto slow access medium.

diagrams whose states are action states. Figure 2-35 is an activity diagram corresponding to the state diagram in Figure 2-31. An alternate and equivalent view of activity diagrams is to interpret action states as control flow between activities and transitions; that is, the arrows are interpreted as sequential constraints between activities.

Decisions are branches in the control flow. They denote alternative transitions based on a condition of the state of an object or a set of objects. Decisions are depicted by a diamond with one or more incoming arrows and two or more outgoing arrows. The outgoing arrows are labeled with the conditions that select a branch in the control flow. The set of all outgoing transitions from a decision represents the set of all possible outcomes. In Figure 2-36, a decision after the OpenIncidentaction selects between three branches: If the incident is of high priority and if it is a fire, the FireChiefis notified. If the incident is of high priority and is not a fire, the ChiefOfPolicés notified. Finally, if neither condition is satisfied, that is, if the Incidentis of low priority, no superior is notified and the resource allocation proceeds.

Complex transitions are transitions with multiple source states or multiple target states. Complex transitions denote the synchronization of multiple activities (in the case of multiple sources) or the splitting of the flow of control into multiple threads (in the case of multiple targets). For example, in Figure 2-37, the action states AllocateResources

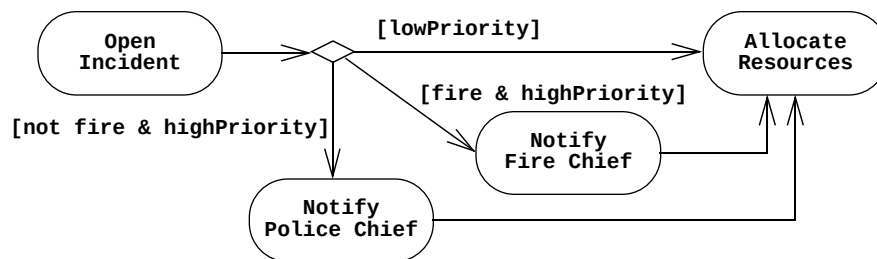


Figure 2-36 Example of decision in the OpenIncident process. If the Incident is a fire and is of high priority, the Dispatcher notifies the FireChief. If it is a high-priority Incident that is not a fire, the PoliceChief is notified. In all cases, the Dispatcher allocates resources to deal with the Incident.

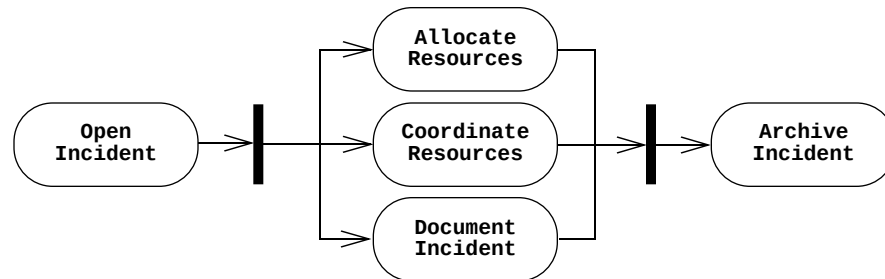


Figure 2-37 An example of complex transitions in a UML activity diagram.

`CoordinateResources` and `DocumentIncident` may all occur in parallel. However, they can only be initiated after the `OpenIncident` action, and the `ArchiveIncident` action may only be initiated after all other activities have been completed.

Actions may be grouped into **swimlanes** to denote the object or subsystem that implements the actions. Swimlanes are represented as rectangles enclosing a group of actions. Transitions may cross swimlanes. In Figure 2-38, the `Dispatcher` swimlane groups all the actions that are performed by the `Dispatcher` object. The `FieldOfficer` swimlane denotes that the `FieldOfficer` object is responsible for the `DocumentIncident` action.

Applying activity diagrams

Activity diagrams provide a task-centric view of the behavior of an object. They can be used, for example, to describe sequencing constraints among use cases, sequential activities among a group of objects, or the tasks of a project. In this book, we use activity diagrams to describe the activities of software development in Chapter 11, *Project Management*, and Chapter 12, *Software Life Cycle*.

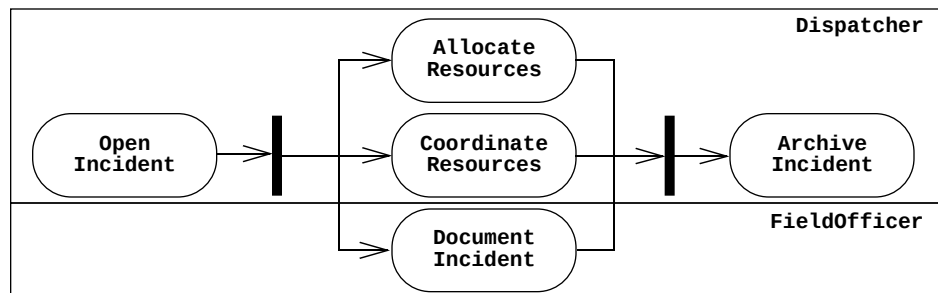


Figure 2-38 An example of swimlanes in a UML activity diagram.

2.4.6 Diagram organization

Models of complex systems quickly become complex as developers refine them. The complexity of models can be dealt with by grouping related elements into **packages**. A package is a grouping of model elements, such as use cases, classes, or activities, defining scopes of understanding.

For example, Figure 2-39 depicts use cases of the FRIEND system, grouped by actors. Packages are displayed as rectangles with a tab attached to their upper-left corner. Use cases dealing with incident management (e.g., creating, resource allocation, documentation) are grouped in the IncidentManagement package. Use cases dealing with incident archive (e.g., archiving an incident, generating reports from archived incidents) are grouped in the IncidentArchive package. Use cases dealing with system administration (e.g., adding users, registering end stations) are grouped in the SysAdministration package. This enables the client and the developers to organize use cases into related groups and to focus on only a limited set of use cases at a time.

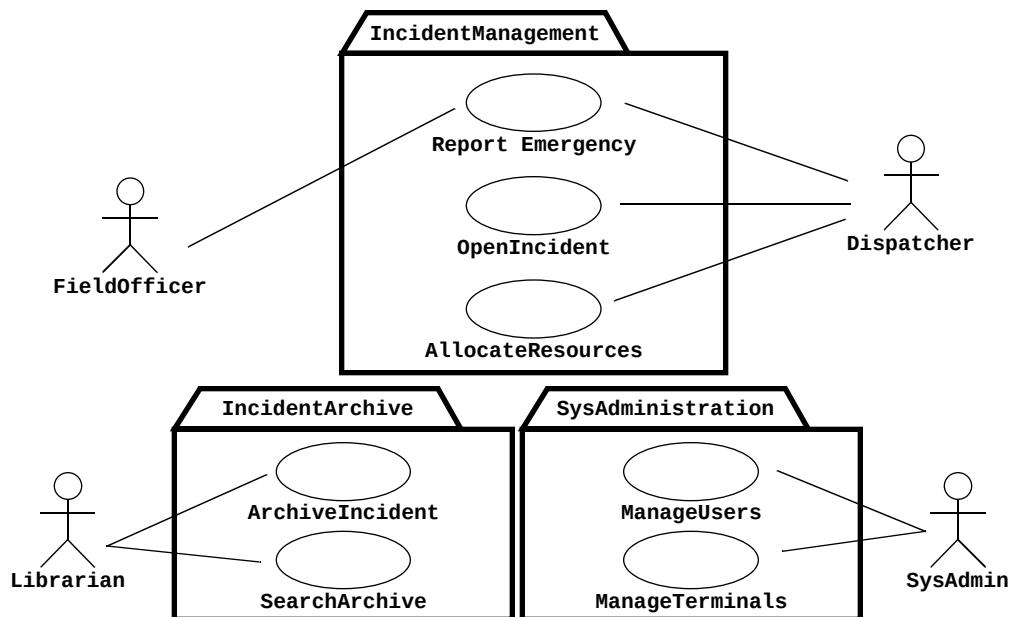


Figure 2-39 Example of packages: use cases of FRIEND organized by actors (UML use case diagram).

Figures 2-39 and 2-40 are examples of class diagrams using packages. Classes from the **ReportEmergency** use case are organized according to the site where objects are created. **FieldOfficer** and **EmergencyReport** are part of the **FieldStation** package, and **Dispatcher** and **Incident** are part of the **DispatcherStation**. Figure 2-39 displays the packages with the

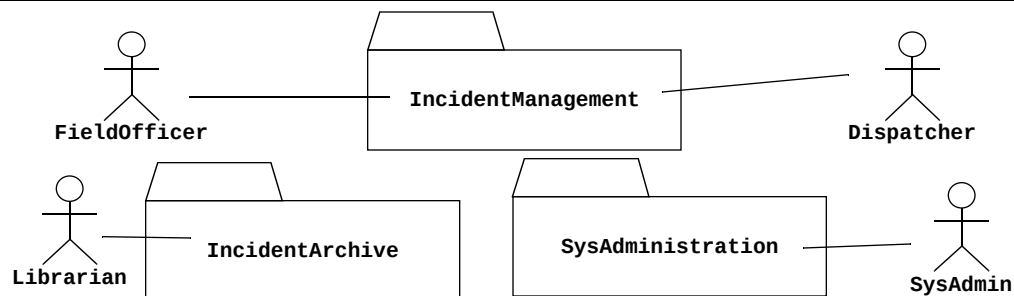


Figure 2-40 Example of packages: This figure displays the same packages as Figure 2-39 except that the details of each package are suppressed (UML use case diagram).

model elements they contain while Figure 2-40 displays the same information without the contents of each package. Figure 2-40 is a higher level picture of the system and can be used for discussing system-level issues, whereas Figure 2-39 is a more detailed view, which can be used to discuss the content of specific packages.

Packages (Figure 2-41) are used to deal with complexity the same way a user organizes files and subdirectories into directories. However, packages are not necessarily hierarchical: The same class may appear in more than one package. To reduce inconsistencies, classes (more generally model elements) are owned by exactly one package, whereas the other packages are said to refer to the modeling element. Note that packages are organizing constructs, not objects. They have no behavior associated with them and cannot send and receive messages.

A **note** is a comment attached to a diagram. Notes are used by developers for attaching information to models and model elements. This is an ideal mechanism for recording outstanding issues relevant to a model, clarifying a complex point, or recording to-dos or reminders. Although notes have no semantics per se, they are sometimes used to express constraints that cannot otherwise be expressed in UML. Figure 2-42 provides an example of a note.

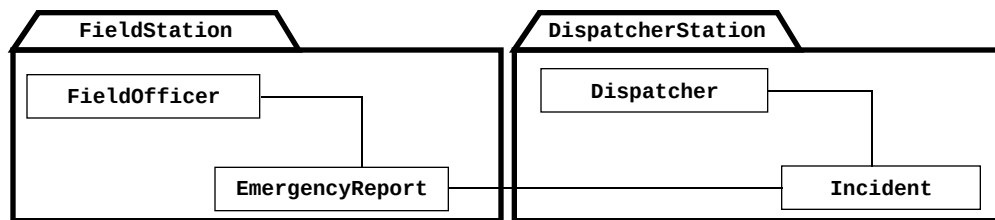


Figure 2-41 Example of packages. The FieldOfficer and EmergencyReport classes are located in the FieldStation package, and the Dispatcher and Incident classes are located on the DispatcherStation package.

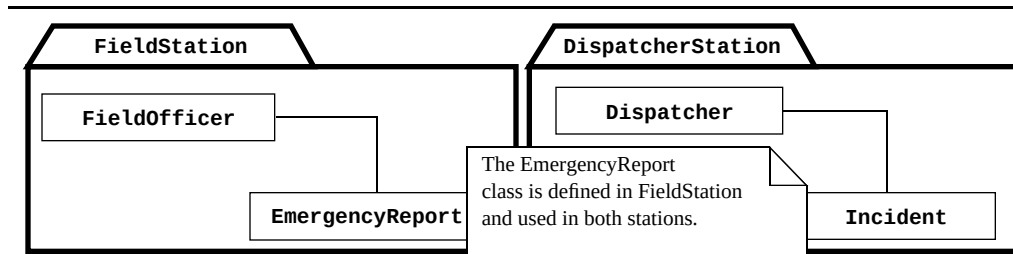


Figure 2-42 An example of a note. Notes can be attached to a specific element in a diagram.

2.4.7 Diagram extensions

The goal of the UML designers is to provide a set of notations to model a broad class of software systems. They also recognized that a fixed set of notations could not achieve this goal, because it is impossible to anticipate the needs encountered in all application and solution domains. For this reason, UML provides a number of extension mechanisms enabling the modeler to extend the language. In this section, we describe two such mechanisms, **stereotypes** and **constraints**.

A **stereotype** is a string enclosed by angle brackets (e.g., `<<subsystem>>`), which is attached to a UML element, such as a class or an association. This enables modelers to create new kinds of building blocks that are needed in their domain. For example, during analysis, we classify objects into three types: entity, boundary, and control. The base UML language knows only objects. To introduce these three additional types, we use three stereotypes, `<<entity>>`, `<<boundary>>`, and `<<control>>` to represent the object type (Figure 2-43). Another example is the relationships among use cases. As we saw in Section 2.4.1, include relationships in use case diagrams are denoted with a dashed arrow and the `<<include>>` stereotype. In this book, we will define the meaning of each stereotype as we introduce them. The `<<entity>>`, `<<boundary>>`, and `<<control>>` stereotypes are described in Chapter 5, *Analysis*.

A **constraint** is a rule that is attached to a UML building block. This allows to represent phenomena that cannot otherwise be expressed with UML. For example, in Figure 2-44, an Incident may be associated with one or more EmergencyReport from the field. From the Dispatchers' perspective, however, it is important that they are able to view the reports chronologically. We represent the chronological ordering of EmergencyReport to Incident

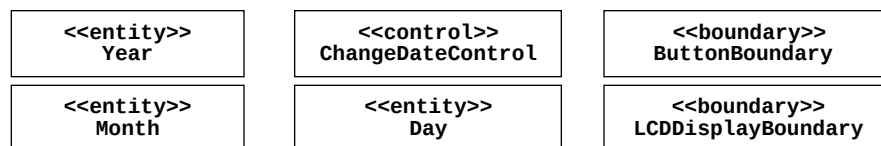


Figure 2-43 Examples of stereotypes (UML class diagram).

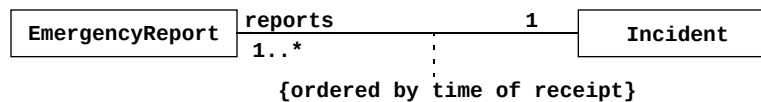


Figure 2-44 An example of constraint (UML class diagram).

associates with the constraint `{ordered by time of receipt}`. Constraints can be expressed as an informal string or by using a formal language such as OCL (Object Constraint Language, [OMG, 1998]). We describe OCL and the use of constraints in Chapter 7, *Object Design*.

2.5 Exercises

1. Draw a use case diagram for a ticket distributor for a train system. The system includes two actors: a traveler, who purchases different types of tickets, and a central computer system, which maintains a reference database for the tariff. Use cases should include: `BuyOneWayTicket`, `BuyWeeklyCard`, `BuyMonthlyCard`, `UpdateTariff`. Also include the following exceptional cases: `Time-Out` (i.e., traveler took too long to insert the right amount), `TransactionAborted` (i.e., traveler selected the cancel button without completing the transaction), `DistributorOutOfChange`, and `DistributorOutOfPaper`.
2. Draw a class diagram representing a book defined by the following statement: "A book is composed of a number of parts, which in turn are composed of a number of chapters. Chapters are composed of sections." Focus only on classes and relationships.
3. Draw an object diagram representing the first part of this book (i.e., Part I, *Getting Started*). Make sure that the object diagram you draw is consistent with the class diagram of Exercise 2.
4. Extend the class diagram of Exercise 2 to include the following attributes:
 - a book includes a publisher, publication date, and an ISBN,
 - a part includes a title and a number,
 - a chapter includes a title, a number, and an abstract, and
 - a section includes a title and a number.
5. Consider the class diagram of Exercise 4. Note that the `Part`, `Chapter`, and `Section` classes all include a title and a number attribute. Add an abstract class and a generalization relationship to factor out these two attributes into the abstract class.
6. Draw a sequence diagram for the `warehouseOnFire` scenario of Figure 2-17. Include the objects `bob`, `alice`, `john`, `FRIEND`, and instances of other classes you may need. Draw only the first five message sends.
7. Draw a sequence diagram for the `ReportIncident` use case of Figure 2-16. Draw only the first five message sends. Make sure it is consistent with the sequence diagram of Exercise 6.

Exercises

8. Consider the software development activities which we described in Section 1.4 in Chapter 1, *Introduction to Software Engineering*. Draw an activity diagram depicting these activities, assuming they are executed strictly sequentially. Draw a second activity diagram depicting the same activities occurring incrementally (i.e., one part of the system is analyzed, designed, implemented, and tested completely before the next part of the system is developed). Draw a third activity diagram depicting the same activities occurring concurrently.

References

- [Booch, 1994] G. Booch, *Object-Oriented Analysis and Design with Applications*, 2nd ed. Benjamin/Cummings, Redwood City, CA, 1994.
- [Coad et al., 1995] P. Coad, D. North, & M. Mayfield, *Object Models: Strategies, Patterns, & Applications*. Prentice Hall, Englewood Cliffs, NJ, 1995.
- [De Marco, 1978] T. De Marco, *Structured Analysis and System Specification*. Yourdon, New York, 1978.
- [FRIEND, 1994] *FRIEND Project Documentation*, School of Computer Science, Carnegie Mellon Univ., Pittsburgh, PA, 1994.
- [Harel, 1987] D. Harel, "Statecharts: A visual formalism for complex systems," *Science of Computer Programming*, pp. 231–274, 1987.
- [Jacobson et al., 1992] I. Jacobson, M. Christerson, P. Jonsson, & G. Overgaard, *Object-Oriented Software Engineering—A Use Case Driven Approach*. Addison-Wesley, Reading, MA, 1992.
- [Liskov & Guttag, 1986] B. Liskov & J. Guttag, *Abstraction and Specification in Program Development*. MIT Press, McGraw-Hill, New York, 1986.
- [Martin & Odell, 1992] J. Martin & J. J. Odell, *Object-Oriented Analysis and Design*. Prentice Hall, Englewood Cliffs, NJ, 1992.
- [Mellor & Shlaer, 1998] S. Mellor & S. Shlaer, *Recursive Design Approach*. Prentice Hall, Upper Saddle River, NJ, 1998.
- [Miller, 1956] G.A. Miller, "The magical number seven, plus or minus two: Some limits on our capacity for processing information," *Psychological Review*, vol. 63, pp. 81–97, 1956.
- [OMG, 1998] Object Management Group, *OMG Unified Modeling Language Specification*. Framingham, MA, 1998. <http://www.omg.org>.
- [Popper, 1992] K. Popper, *Objective Knowledge: An Evolutionary Approach*. Clarendon, Oxford, 1992.
- [Rumbaugh et al., 1991] J. Rumbaugh, M. Blaha, W. Premerlani, F. Eddy, and W. Lorensen. *Object-Oriented Modeling and Design*. Prentice Hall, Englewood Cliffs, NJ, 1991.
- [Wirfs-Brock et al., 1990] R. Wirfs-Brock, B. Wilkerson, & L. Wiener, *Designing Object-Oriented Software*. Prentice Hall, Englewood Cliffs, NJ, 1990.

3

3.1	Introduction: A rocket example	64
3.2	An overview of project communication	65
3.3	Modes of communication	66
3.3.1	Problem definition	68
3.3.2	Client reviews	69
3.3.3	Project reviews	70
3.3.4	Inspections and walkthroughs	70
3.3.5	Status reviews	71
3.3.6	Brainstorming	71
3.3.7	Releases	72
3.3.8	Postmortem review	72
3.3.9	Request for clarification	73
3.3.10	Request for change	74
3.3.11	Issue resolution	74
3.3.12	Discussion	76
3.4	Mechanisms of communication	76
3.4.1	Hallway conversations	78
3.4.2	Questionnaires and structured interviews	78
3.4.3	Meetings	79
3.4.4	Reviews	80
3.4.5	Same-time, different-place groupware	82
3.4.6	Electronic mail	83
3.4.7	Newsgroups	83
3.4.8	World Wide Web	84
3.4.9	Lotus Notes	84
3.4.10	Discussion	86
3.5	Project communication activities	87
3.5.1	Identifying communication needs	87
3.5.2	Setting up an infrastructure	88
3.5.3	Organizing client and project reviews	89
3.5.4	Organizing weekly team meetings	91
3.5.5	Transition issues revisited	93
3.6	Exercises	93
	References	94